

Resolución de problemas enlazando distintas estructuras

Algoritmos y Estructuras de Datos

2^{do} cuatrimestre 2026

- ▶ Vimos distintas estructuras de datos, las cuales nos permiten modelar TADs clásicos.
- ▶ Distintas estructuras de datos nos permiten resolver problemas de forma eficiente.
- ▶ Pero a veces no alcanza con una sola estructura.

- ▶ Vimos distintas estructuras de datos, las cuales nos permiten modelar TADs clásicos.
- ▶ Distintas estructuras de datos nos permiten resolver problemas de forma eficiente.
- ▶ Pero a veces no alcanza con una sola estructura.
- ▶ ¿Qué pasa si quiero acceder a datos según múltiples criterios?

Ejemplo 1

Una empresa tiene un deposito al que le llegan pedidos. Estos pedidos vienen de distintos clientes, pidiendo productos del depósito, y cuentan con un número identificador. Queremos un sistema que nos permita administrar los pedidos, para ir cumpliéndolos en orden de llegada. Además, necesitamos poder ver la información de los pedidos.

Ejemplo 1 - TAD

```
Pedido ES struct<id:N,  
                cliente:N,  
                producto:N>  
TAD SistemaPedidos {  
    obs cola:seq<Pedido>  
  
    proc nuevoSistema() : SistemaPedidos  
        requiere ...  
        asegura ...  
    proc agregarPedido(inout s: SistemaPedidos,  
                      in p: Pedido)  
    proc proximo(inout s: SistemaPedidos) : Pedido  
    proc infoPedido(in s: SistemaPedidos,  
                   in idPedido: N): Pedido  
}
```

Ejemplo 1 - Clase Java

```
class SistemaPedidos {  
    SistemaPedidos() {...}  
    void agregarPedido(Pedido p) {...}  
    Pedido proximo() {...}  
    Pedido infoPedido(int idPedido) {...}  
}
```

Ejemplo 1 - Estructura

Elijamos una estructura interna, pensando en las complejidades.

Ejemplo 1 - Estructura

Elijamos una estructura interna, pensando en las complejidades.

```
class SistemaPedidos {  
    ColaSobreLista<Pedido> colaDePedidos;  
    ...  
}
```

- ▶ SistemaPedidos()
- ▶ void agregarPedido(Pedido p)
- ▶ Pedido proximo()
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Estructura

Elijamos una estructura interna, pensando en las complejidades.

```
class SistemaPedidos {  
    ColaSobreLista<Pedido> colaDePedidos;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p)
- ▶ Pedido proximo()
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Estructura

Elijamos una estructura interna, pensando en las complejidades.

```
class SistemaPedidos {  
    ColaSobreLista<Pedido> colaDePedidos;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(1)$
- ▶ Pedido proximo()
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Estructura

Elijamos una estructura interna, pensando en las complejidades.

```
class SistemaPedidos {  
    ColaSobreLista<Pedido> colaDePedidos;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(1)$
- ▶ Pedido proximo() $\rightarrow O(1)$
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Estructura

Elijamos una estructura interna, pensando en las complejidades.

```
class SistemaPedidos {  
    ColaSobreLista<Pedido> colaDePedidos;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(1)$
- ▶ Pedido proximo() $\rightarrow O(1)$
- ▶ Pedido infoPedido(int idPedido) $\rightarrow O(n)!!$

Ejemplo 1 - Estructura

Elijamos una estructura interna, pensando en las complejidades.

```
class SistemaPedidos {  
    ColaSobreLista<Pedido> colaDePedidos;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(1)$
- ▶ Pedido proximo() $\rightarrow O(1)$
- ▶ Pedido infoPedido(int idPedido) $\rightarrow O(n)!!$

¿Podemos mejorar infoPedido?

Ejemplo 1 - Estructura

```
class SistemaPedidos {  
    // Usamos la cola para resolver los pedidos en orden  
    ColaSobreLista<Pedido> colaDePedidos;  
    // Y guardamos los pedidos tambien en un diccionario  
    // para acceder a su info  
    DiccionarioLog<int, Pedido> diccPedidosPorId;  
    ...  
}
```

Ejemplo 1 - Estructura

```
class SistemaPedidos {  
    ColaSobreLista<Pedido> colaDePedidos;  
    DiccionarioLog<int, Pedido> diccPedidosPorId;  
    ...  
}
```

- ▶ SistemaPedidos()
- ▶ void agregarPedido(Pedido p)
- ▶ Pedido proximoPedidoPorLlegada()
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Estructura

```
class SistemaPedidos {  
    ColaSobreLista<Pedido> colaDePedidos;  
    DiccionarioLog<int, Pedido> diccPedidosPorId;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p)
- ▶ Pedido proximoPedidoPorLlegada()
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Estructura

```
class SistemaPedidos {  
    ColaSobreLista<Pedido> colaDePedidos;  
    DiccionarioLog<int, Pedido> diccPedidosPorId;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(\log(n))$
- ▶ Pedido proximoPedidoPorLlegada()
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Estructura

```
class SistemaPedidos {  
    ColaSobreLista<Pedido> colaDePedidos;  
    DiccionarioLog<int, Pedido> diccPedidosPorId;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(\log(n))$
- ▶ Pedido proximoPedidoPorLlegada() $\rightarrow O(\log(n))$
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Estructura

```
class SistemaPedidos {  
    ColaSobreLista<Pedido> colaDePedidos;  
    DiccionarioLog<int, Pedido> diccPedidosPorId;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(\log(n))$
- ▶ Pedido proximoPedidoPorLlegada() $\rightarrow O(\log(n))$
- ▶ Pedido infoPedido(int idPedido) $\rightarrow O(\log(n))$

Ejemplo 1 - Estructura

```
class SistemaPedidos {  
    ColaSobreLista<Pedido> colaDePedidos;  
    DiccionarioLog<int, Pedido> diccPedidosPorId;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(\log(n))$
- ▶ Pedido proximoPedidoPorLlegada() $\rightarrow O(\log(n))$
- ▶ Pedido infoPedido(int idPedido) $\rightarrow O(\log(n))$

Algunas operaciones aumentaron su complejidad.

¿Vale la pena? Depende.

Ejemplo 1 - Otra estructura

Ahora del deposito nos piden que se puedan despachar los pedidos por ID del pedido ademas del orden de llegada, ya que algunos pedidos pagan por envío prioritario.
¿Qué estructuras deberiamos utilizar?

Ejemplo 1 - Otra estructura

```
class SistemaPedidos {  
    ArrayList<Pedido> pedidosPorId  
    ColaSobreLista<Pedido> pedidosPorLlegada  
    ...  
}
```

- ▶ SistemaPedidos()
- ▶ void agregarPedido(Pedido p)
- ▶ Pedido proximoPedidoPorLlegada()
- ▶ Pedido proximoPedidoPorId()
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Otra estructura

```
class SistemaPedidos {  
    ArrayList<Pedido> pedidosPorId  
    ColaSobreLista<Pedido> pedidosPorLlegada  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p)
- ▶ Pedido proximoPedidoPorLlegada()
- ▶ Pedido proximoPedidoPorId()
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Otra estructura

```
class SistemaPedidos {  
    ArrayList<Pedido> pedidosPorId  
    ColaSobreLista<Pedido> pedidosPorLlegada  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(1)$
- ▶ Pedido proximoPedidoPorLlegada()
- ▶ Pedido proximoPedidoPorId()
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Otra estructura

```
class SistemaPedidos {  
    ArrayList<Pedido> pedidosPorId  
    ColaSobreLista<Pedido> pedidosPorLlegada  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(1)$
- ▶ Pedido proximoPedidoPorLlegada() $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorId()
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Otra estructura

```
class SistemaPedidos {  
    ArrayList<Pedido> pedidosPorId  
    ColaSobreLista<Pedido> pedidosPorLlegada  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(1)$
- ▶ Pedido proximoPedidoPorLlegada() $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorId() $\rightarrow O(n)$
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Otra estructura

```
class SistemaPedidos {  
    ArrayList<Pedido> pedidosPorId  
    ColaSobreLista<Pedido> pedidosPorLlegada  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(1)$
- ▶ Pedido proximoPedidoPorLlegada() $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorId() $\rightarrow O(n)$
- ▶ Pedido infoPedido(int idPedido) $\rightarrow O(n)$

Ejemplo 1 - Otra estructura

```
class SistemaPedidos {  
    ArrayList<Pedido> pedidosPorId  
    ColaSobreLista<Pedido> pedidosPorLlegada  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(1)$
- ▶ Pedido proximoPedidoPorLlegada() $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorId() $\rightarrow O(n)$
- ▶ Pedido infoPedido(int idPedido) $\rightarrow O(n)$

¿Qué cambia si pedidosPorId es una lista ordenada? ¿ qué complejidades se modifican?

Ejemplo 1 - Otra estructura

```
class SistemaPedidos {  
    ArrayList<Pedido> pedidosPorId  
    ColaSobreLista<Pedido> pedidosPorLlegada  
    ...  
}
```

- ▶ SistemaPedidos()
- ▶ void agregarPedido(Pedido p)
- ▶ Pedido proximoPedidoPorLlegada()
- ▶ Pedido proximoPedidoPorId()
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Otra estructura

```
class SistemaPedidos {  
    ArrayList<Pedido> pedidosPorId  
    ColaSobreLista<Pedido> pedidosPorLlegada  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p)
- ▶ Pedido proximoPedidoPorLlegada()
- ▶ Pedido proximoPedidoPorId()
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Otra estructura

```
class SistemaPedidos {  
    ArrayList<Pedido> pedidosPorId  
    ColaSobreLista<Pedido> pedidosPorLlegada  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorLlegada()
- ▶ Pedido proximoPedidoPorId()
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Otra estructura

```
class SistemaPedidos {  
    ArrayList<Pedido> pedidosPorId  
    ColaSobreLista<Pedido> pedidosPorLlegada  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorLlegada() $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorId()
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Otra estructura

```
class SistemaPedidos {  
    ArrayList<Pedido> pedidosPorId  
    ColaSobreLista<Pedido> pedidosPorLlegada  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorLlegada() $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorId() $\rightarrow O(n)$
- ▶ Pedido infoPedido(int idPedido)

Ejemplo 1 - Otra estructura

```
class SistemaPedidos {  
    ArrayList<Pedido> pedidosPorId  
    ColaSobreLista<Pedido> pedidosPorLlegada  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorLlegada() $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorId() $\rightarrow O(n)$
- ▶ Pedido infoPedido(int idPedido) $\rightarrow O(\log(n))$

Ejemplo 1 - Invariante de Representación

¿Qué debe cumplir la estructura interna en todo momento?

Ejemplo 1 - Invariante de Representación

¿Qué debe cumplir la estructura interna en todo momento?

- ▶ Los pedidos en pedidosPorLlegada y pedidosPorId no tienen ids repetidos.
- ▶ Los pedidos en pedidosPorLlegada y los valores de pedidosPorId son los mismos.
- ▶ Los pedidos en pedidosPorId están ordenados por ID

Ejemplo 1 - Código

```
public class SistemaPedidos {
    ArrayList<Pedido> pedidosPorId;
    ListaEnlazada<Pedido> pedidosPorLlegada;

    public SistemaPedidos() {
        pedidosPorId = new ArrayList<Pedido>();
        pedidosPorLlegada = new ListaEnlazada<Pedido>();
    }

    public void agregar(Pedido p) {
        pedidosPorLlegada.agregarAtras(pedido);
        agregarOrdenado(pedido, pedidosPorID);
    }

    ...
}
```

Ejemplo 1 - Código

```
public class SistemaPedidos {
    ArrayList<Pedido> pedidosPorId;
    ListaEnlazada<Pedido> pedidosPorLlegada;
    ...

    public Pedido proximoPedidoPorId(){
        Pedido pedido = pedidosPorId.get(pedidosPorId.size
        ()-1);
        pedidosPorId.remove(pedido);
        pedidosPorLlegada.remove(pedido);
        return pedido;
    }

    public Pedido proximoPedidoPorLlegada(){
        Pedido pedido = pedidosPorLlegada.obtener(0);
        pedidosPorLlegada.eliminar(0)
        int i = busquedaBinariaIndice(pedidosPorId,
        pedido);
        pedidosPorId.remove(i);
        return pedido;
    }
}
```

- ▶ Vimos que a veces conviene guardar los datos en múltiples lugares.
- ▶ ¡Pero a veces eso tampoco alcanza!
- ▶ Caso más común: queremos acceder rápidamente a un lugar que la estructura sola no nos permite.

Ejemplo 2

El deposito puede tomarse su tiempo, por lo que le da la opción a los clientes de cancelar pedidos.

Ejemplo 2

El deposito puede tomarse su tiempo, por lo que le da la opción a los clientes de cancelar pedidos.

```
TAD SistemaPedidos {  
    ...  
    proc cancelarPedido(inout s: SistemaPedidos,  
                        in id: IN) : Pedido  
}
```

Ejemplo 2

El deposito puede tomarse su tiempo, por lo que le da la opción a los clientes de cancelar pedidos.

```
TAD SistemaPedidos {  
    ...  
    proc cancelarPedido(inout s: SistemaPedidos,  
                        in id: IN) : Pedido  
}
```

¿Qué problema surge con nuestra estructura?

Ejemplo 2

El deposito puede tomarse su tiempo, por lo que le da la opción a los clientes de cancelar pedidos.

```
TAD SistemaPedidos {  
  ...  
  proc cancelarPedido(inout s: SistemaPedidos,  
                      in id: IN) : Pedido  
}
```

¿Qué problema surge con nuestra estructura? Eliminar un elemento arbitrario de una lista enlazada es lento :(

Ejemplo 2

- ▶ Nos gustaría poder “llegar rápido” a un elemento del medio de la lista.

Ejemplo 2

- ▶ Nos gustaría poder “llegar rápido” a un elemento del medio de la lista.
- ▶ ¿Podemos guardarnos referencias a los nodos?

Ejemplo 2

- ▶ Nos gustaría poder “llegar rápido” a un elemento del medio de la lista.
- ▶ ¿Podemos guardarnos referencias a los nodos?
- ▶  NO, ¡romperíamos el encapsulamiento!

Ejemplo 2

- ▶ Nos gustaría poder “llegar rápido” a un elemento del medio de la lista.
- ▶ ¿Podemos guardarnos referencias a los nodos?
- ▶  NO, ¡romperíamos el encapsulamiento!
- ▶ Pero, parecido a los iteradores, podemos encapsular una referencia a un nodo.

- ▶ Un Handle (del inglés para “manija”) es una **referencia encapsulada** a un recurso de la estructura interna de otro sistema.
- ▶ Lo implementaremos como una clase anidada de la estructura correspondiente que, en su representación interna, guarda una referencia al objeto de interés.
- ▶ Sus atributos serán privados, podría o no tener métodos públicos. De tenerlos, éstos deberían llamar a un método privado de su estructura correspondiente.
- ▶ Vamos a **cambiar la interfaz** de nuestra clase para que use Handles.

```
interface Handle<T> {  
    // Devuelve el valor del elemento  
    public T valor();  
  
    //Elimina al elemento de la coleccion  
    public void eliminar();  
}
```

Handles - Implementación

```
public class ListaEnlazada<T> {  
    ...  
  
    private class Handle implements Handle<T>{  
        private Nodo nodo;  
  
        private Handle(Nodo n) {  
            this.nodo = n;  
        }  
        public T valor(){  
            return this.nodo.dato;  
        }  
        public void eliminar() {  
            //...  
        }  
    }  
    ...  
}
```

Handles - Implementación

```
public class ListaEnlazada<T> {  
    ...  
  
    private class Handle {  
        private Nodo nodo;  
  
        private Handle(Nodo n) {  
            this.nodo = n;  
        }  
        public T valor(){  
            return this.nodo.dato;  
        }  
        public void eliminar() {  
            ListaEnlazada.this.eliminarNodo(this.nodo);  
        }  
    }  
    ...  
    // Con el handle, nos evitamos el costo lineal de  
    // buscar el nodo  
    private void eliminarNodo(Nodo h) {...}  
}
```

Ahora que podemos guardarnos referencias de los elementos de la cola:

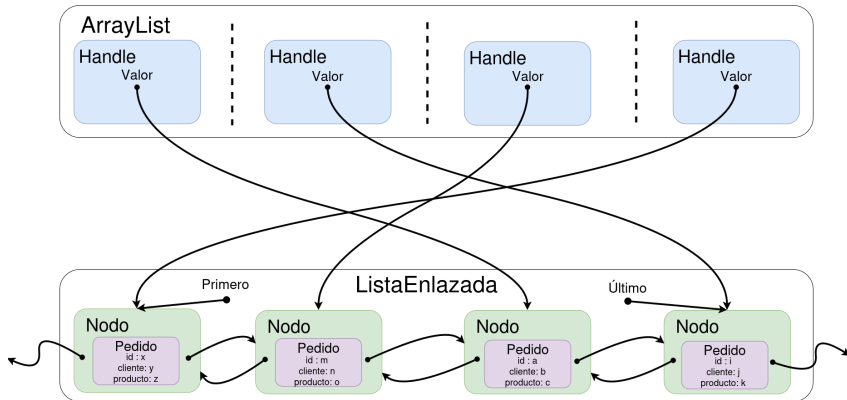
¿Cómo modificamos nuestro diseño para eliminar más rápido?

```
class SistemaPedidos {  
    ArrayList<Pedido> pedidosPorId;  
    ListaEnlazada<Pedido> pedidosPorLlegada;  
    // ...  
}
```

Podemos armar el ArrayList con los Handles en vez de los pedidos

```
class SistemaPedidos {  
    ArrayList<Handle<Pedido>> pedidosPorId;  
    ListaEnlazada<Pedido> pedidosPorLlegada;  
    //...  
}
```

Handles - Diagrama



Ejemplo 2 - Costos Temporales

```
class SistemaPedidos {  
    ArrayList<Handle<Pedido>> pedidosPorId;  
    ListaEnlazada<Pedido> pedidosPorLlegada;  
    ...  
}
```

- ▶ SistemaPedidos()
- ▶ void agregarPedido(Pedido p)
- ▶ Pedido proximoPedidoPorLlegada()
- ▶ Pedido proximoPedidoPorId()
- ▶ Pedido cancelarPedido(int id)

Ejemplo 2 - Costos Temporales

```
class SistemaPedidos {  
    ArrayList<Handle<Pedido>> pedidosPorId;  
    ListaEnlazada<Pedido> pedidosPorLlegada;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p)
- ▶ Pedido proximoPedidoPorLlegada()
- ▶ Pedido proximoPedidoPorId()
- ▶ Pedido cancelarPedido(int id)

Ejemplo 2 - Costos Temporales

```
class SistemaPedidos {  
    ArrayList<Handle<Pedido>> pedidosPorId;  
    ListaEnlazada<Pedido> pedidosPorLlegada;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorLlegada()
- ▶ Pedido proximoPedidoPorId()
- ▶ Pedido cancelarPedido(int id)

Ejemplo 2 - Costos Temporales

```
class SistemaPedidos {  
    ArrayList<Handle<Pedido>> pedidosPorId;  
    ListaEnlazada<Pedido> pedidosPorLlegada;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorLlegada() $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorId()
- ▶ Pedido cancelarPedido(int id)

Ejemplo 2 - Costos Temporales

```
class SistemaPedidos {  
    ArrayList<Handle<Pedido>> pedidosPorId;  
    ListaEnlazada<Pedido> pedidosPorLlegada;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorLlegada() $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorId() $\rightarrow O(1)$
- ▶ Pedido cancelarPedido(int id)

Ejemplo 2 - Costos Temporales

```
class SistemaPedidos {  
    ArrayList<Handle<Pedido>> pedidosPorId;  
    ListaEnlazada<Pedido> pedidosPorLlegada;  
    ...  
}
```

- ▶ SistemaPedidos() $\rightarrow O(1)$
- ▶ void agregarPedido(Pedido p) $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorLlegada() $\rightarrow O(n)$
- ▶ Pedido proximoPedidoPorId() $\rightarrow O(1)$
- ▶ Pedido cancelarPedido(int id) $\rightarrow O(\log(n))$

Ya se encuentra subido el taller 5 - estructuras enlazadas y handles en el Campus.

Fecha *límite* de entrega: 11/10